

19. Intégrer une carte dans son projet

Une carte dessinée à la souris est une fonction du programme : on l'appelle, et le décor apparaît.

1. Ce qu'il faut comprendre

Ce programme a les trois sortes de dessins du jeu : une **tuile de 8 × 8** (`Tuile ARBRE`), un **personnage de 16 × 16** (`Perso HEROS` — quatre tuiles d'un coup), et une **carte** (`VILLAGE`, tout un écran). Les deux premiers se peignent dans  **Les tuiles** ; la carte, dans  **La carte**. Essaie sur chacun les palettes, la sélection et le déplacement.

Une **carte**, c'est un décor entier dessiné à la souris, dans l'onglet  **La carte**. L'atelier ne la garde pas de son côté : il **l'écrit dans ton programme**, sous la forme d'une **fonction** — ici `void VILLAGE()`, entre deux commentaires `/* --- carte "VILLAGE" --- */` qui la marquent. C'est grâce à eux qu'il la retrouve et la réécrit quand tu dessines.

La créer : onglet  **La carte**, « + carte », et un nom en majuscules — VILLAGE, MAISON, NIVEAU1... Puis dessine : chaque carré de 8 × 8 que tu remplis devient une ligne `poser(colonne, ligne, {...})`, avec ses huit rangées. Un carré laissé vide n'est pas écrit : il ne coûte rien.

Lire le code de la carte : dans `poser(9, 10, {...})`, **9 est la colonne** (0 tout à gauche, 19 tout à droite) et **10 la ligne** (0 tout en haut, 17 tout en bas). Une case fait 8 × 8 pixels : la case (9, 10) commence donc au pixel $x = 72$, $y = 80$. Les **huit textes entre accolades** sont les huit rangées de pixels du carré, de haut en bas, et **chaque chiffre est un pixel** : 0 le plus clair, 1, 2, puis 3 le plus sombre. Regarde le toit, case (9, 9) : ses « 3 » dessinent la pente, rangée après rangée. En couleur, une ligne `teindre(colonne, ligne, palette)` dit en plus quelle palette prend la case.

Lire la carte pendant le jeu : `lire(colonne, ligne)` rend le **numéro de la tuile** affichée à cet endroit — 0 si la case est vide. C'est ainsi qu'un jeu sait si le joueur arrive sur un mur, sur le sol ou dans le vide. Le programme le montre en bas de l'écran : il relit la case (9, 11), du sol, et l'écrit avec `nombre()` ; la case (5, 11), vide, donne 0.

Une fonction ne fait rien tant qu'on ne l'appelle pas. C'est l'oubli le plus fréquent : la carte est dessinée, le programme compile, et l'écran reste vide. Il manque `VILLAGE();` dans `main`, **avant** la boucle du jeu — une seule fois : un décor reste à l'écran tant qu'on ne l'efface pas.

Essaie : mets `//` devant `VILLAGE();`. Le titre reste, la maison disparaît. Enlève-le : elle revient. Tout le décor tient dans cette seule ligne.

Plusieurs cartes : chacune a sa fonction. Pour passer de l'une à l'autre, on efface puis on appelle l'autre : `effacer();` puis `MAISON();`. Les huit carrés du sol, identiques, ne prennent **qu'une tuile** : le compilateur reconnaît un dessin qu'il a déjà.

Copier, déplacer : dans l'atelier de la carte, l'outil  **Sélection** choisit une zone — un rectangle **libre** tracé à la souris, ou d'un clic **un carré de 8 × 8** ou **un bloc de 16 × 16**, calés sur la grille. **Attrape** la zone et **glisse-la** : elle se déplace, et retombe sur la case la plus proche ; sa place redevient le fond. **Ctrl+C**, **Ctrl+X** et **Ctrl+V** la copient, la coupent et la collent. Dans l'atelier des tuiles,  **Sélectionner et déplacer** fait de même à l'intérieur d'une tuile de 8 × 8 ou d'un personnage de 16 × 16, et  **Copier le dessin**,  **Coller ici** et  **Dupliquer** copient un dessin entier. Chaque geste s'annule avec **Ctrl+Z**.

Un jeu, sans écrire la boucle : dans 🦿 La carte, « ► Jouer la scène » écrit `main` pour toi — la carte, les murs, le joueur qui bouge, les acteurs et les événements. Tes propres lignes de `main` sont alors remplacées ; l'atelier demande avant de le faire.

Poser une tuile, poser un personnage : `poser(6, 10, ARBRE);` met l'arbre dans la case (6, 10), et `teindre(6, 10, 1);` lui donne la palette 1 — celle de la forêt. `sprite16(0, 60, 72, HEROS);` pose le personnage **au pixel près**, en x = 60, y = 72 : un personnage n'est pas sur la grille, il se déplace librement par-dessus le décor.

À toi : descends à l'atelier de la carte, dessine un arbre à côté de la maison, et regarde un nouveau `poser(...)` apparaître dans `VILLAGE()`.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2138 octets** de cartouche.

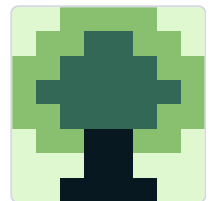
1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
/* Une TUILE de 8 × 8 : un arbre. Elle se peint dans 🧱 Les tuiles. */
```

2. UN DESSIN — ARBRE

```
Tuile ARBRE = {
  "00111100",
  "01122110",
  "11222211",
  "12222221",
  "11222211",
  "01133110",
  "00033000",
  "00333300",
};

/* Un PERSONNAGE de 16 × 16 : quatre tuiles de 8 × 8 d'un coup. */
```



à l'écran

3. UN DESSIN — HEROS

```
Perso HEROS = {
  "0000033333300000",
  "0000322222230000",
  "000322222223000",
  "0003211221123000",
  "0003211221123000",
  "000322222223000",
  "0000322332230000",
  "0000033333300000",
  "000333333333000",
  "003222222222300",
  "032122222221230",
  "032122222221230",
  "000322222223000",
  "0003322003223000",
  "0003220000223000",
  "0033330000333300",
};

/* --- carte "VILLAGE" --- */
```



à l'écran

4. UNE FONCTION — VILLAGE()

```
void VILLAGE() {
  poser(9, 9, {
    "00000033",
    "00003322",
    "00332222",
    "33222222",
    "32222222",
    "33333333",
    "31111113",
    "31111113",
  });
  poser(10, 9, {
    "33000000",
    "22330000",
    "22223300",
    "22222233",
    "22222223",
    "33333333",
    "31111113",
    "31111113",
  });
  poser(9, 10, {
    "31111113",
    "31133113",
    "31133113",
    "31111113",
    "31111333",
    "31111311",
    "31111311",
    "33333311",
  });
  poser(10, 10, {
    "31111113",
    "31111113",
    "33311113",
    "11311113",
    "11311113",
    "11311113",
    "11311113",
    "11333333",
  });
  poser(6, 11, {
    "33333333",
    "21212121",
    "12121212",
    "21212121",
    "12121212",
    "21212121",
    "12121212",
    "21212121",
  });
  poser(7, 11, {
    "33333333",
    "21212121",
    "12121212",
    "21212121",
    "12121212",
    "21212121",
    "12121212",
    "21212121",
  });
  poser(8, 11, {
    "33333333",
    "21212121",
```

```

"12121212",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
});
poser(9, 11, {
"33333333",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
});
poser(10, 11, {
"33333333",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
});
poser(11, 11, {
"33333333",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
});
poser(12, 11, {
"33333333",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
});
poser(13, 11, {
"33333333",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
"12121212",
"21212121",
});
}
/* --- fin de la carte "VILLAGE" --- */

```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    /* Les couleurs : la palette 1 pour l'arbre (une forêt). */
    couleurFond(1, 0, 22, 31, 14);
    couleurFond(1, 1, 10, 24, 6);
    couleurFond(1, 2, 4, 14, 2);
    couleurFond(1, 3, 12, 6, 2);

    texte(4, 3, "MON VILLAGE");

    VILLAGE();    // la carte : SANS CET APPEL, RIEN NE S'AFFICHE

    /* La tuile ARBRE, posée à côté de la maison, dans la palette 1. */
    poser(6, 10, ARBRE);
    teindre(6, 10, 1);
    poser(13, 10, ARBRE);
    teindre(13, 10, 1);

    /* Le personnage HEROS, au pixel près : un lutin de 16 × 16. */
    sprite16(0, 60, 72, HEROS);

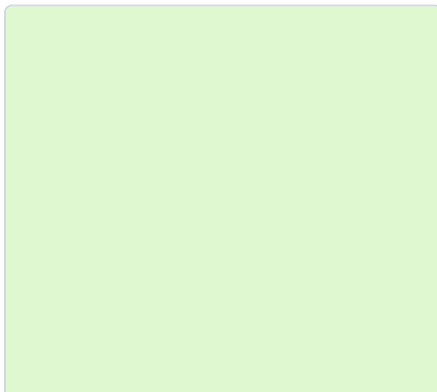
    /* Relire la carte : le numéro de la tuile à une case, 0 si elle est vide. */
    texte(1, 15, "SOL");
    nombre(6, 15, lire(9, 11));
    texte(1, 16, "VIDE");
    nombre(6, 16, lire(3, 11));

    while (true) {
        image();
    }

    return 0;
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Le titre « MON VILLAGE », la maison sur huit carrés de sol, un arbre de chaque côté (dans la palette de la forêt), le personnage devant la maison, et en bas « SOL 053 » et « VIDE 000 ».

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le titre est écrit
- ✓ la carte : la maison et ses quatre carrés
- ✓ le sol, sur huit cases, ne prend qu'une tuile
- ✓ la tuile ARBRE (8 × 8) est posée deux fois, la même
- ✓ dans la palette 1, par `teindre()`
- ✓ le personnage HEROS (16 × 16) est à l'écran, au pixel près ($x = 60$, $y = 72$)
- ✓ `lire(9, 11)` relit la tuile du sol (053)
- ✓ `lire(3, 11)`, une case vide, donne 000

Ensuite : **20. Les couleurs : palettes, variétés et thèmes** — niveau 3 sur 10. · `gameboy3` — écrire une cartouche Game Boy en C++